

Relatório de Algoritmo de Análise de Algoritmo em C.

Projeto: Analisador de Algoritmos de Ordenação

Aluno: William dos Santos Ferreira

1. Introdução

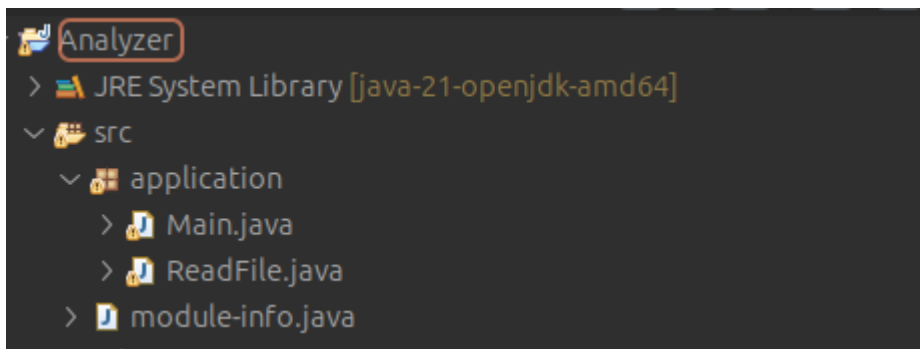
O projeto tem como objetivo realizar análises de algoritmos de ordenação (BubbleSort, QuickSort, MergeSort, entre outros), ele em si, lê um arquivo de código ou texto na linguagem de C e a faz uma tanto análise assintótica (em casos que o a complexidade seja $O(n)$ ou $O(n^k)$, quanto a complexidade assintótica (em casos de algoritmos que tenham complexidade $O(n \log n)$). Para a análise é considerado loops, declarações de variáveis, linhas de comando, recursão.

O código em si fornece informações relacionado ao consumo de tempo do código, em sua análise, que é baseado em modelos de computação RAM, no qual idealiza que um computador tem memória infinita, cada ação, excluindo loops e e recursões, é considerado trabalho +1 para o analisador. O objetivo final é analisarmos códigos e enxergamos seu custo de tempo, o algoritmo foi produzido utilizando a linguagem JAVA.

2. Projeto

2.1 Árvore arquivos

O projeto possui uma pasta **src** que leva ao módulo **application** e que nele temos duas classes, a **Main**, que é a principal, responsável pela entrada de dados e a classe chamada **ReadFile**, esta, é responsável pela leitura dos dados, e demais funcionalidades. Ademais, temos uma pasta no projeto chamada **Files**, este diretório é aonde iremos inserir os código que desejamos que nossa aplicação análise, contudo, pode ser configurado para encontrar códigos em outro diretórios, basta inserirmos informações na entrada de dados.



2.2 Funcionamento

Nossa aplicação se inicia chamando a classe **main**, que, logo no início, irá mostrar uma mensagem pedindo para o usuário inserir o diretório do arquivo que ele deseja realizar análise, com uma sugestão de como escrever o diretório desejado:

```
Scanner input = new Scanner(System.in);
System.out.println("Digite o nome do diretório e do arquivo\nExemplo: './Files/nomeArquivo'");
String str = input.next();
```

```
Digite o nome do diretório e do arquivo
Exemplo: './Files/nomeArquivo'
./Files/apresentacao.c
```

Após inserirmos o diretório, instanciamos um objeto do tipo `ReadFile`, classe no qual é responsável pela leitura do arquivo e análise dos dados em si:

```
ReadFile reader = new ReadFile(new File(str));
```

O objeto `reader` possuirá as seguintes funções:

- `Read()` - Responsável por ler o arquivo, além de chamar outra função que realiza análise;
- `Analyzer()` - Função que pega linha por linha do arquivo, transforma em string e realiza a análise em si;
- `GetFunctionName()` - Função responsável por analisar string por string da linha do arquivo, para salvar nomes de funções e comparar se nela há uma recursão;
- `Result()` - Responsável por imprimir o resultado na tela para o usuário;

Além de possuir os demais atributos:

- `MyFile` - do tipo `File` (Arquivo), que é o arquivo que usuário deseja analisar;
- `N` - do tipo `int`, referente a quantidade de loops;
- `K` - do tipo `int`, quantidade de trabalho;
- `Recursion` - do tipo `int`, quantidade de recursões no código;
- `NElevated` - do tipo `int`, verifica se a quantidade de um Loop dentro de um loop;

- OpenLoop - do tipo int, itera a abertura de um loop;
- CloseLoop - do tipo int, itera o fechamento de um loop;
- RecursionBool - do tipo boolean, Define se há uma recursão no projeto;
- NameFunction - do tipo string, Armazena um nome de uma função para comparar se há ela é chamada novamente dentro dela;
- LastNameFunction - do tipo string, Armazena o último nome de função que foi registrado até fazermos análise da próxima linha do arquivo;

```

1 package application;
2
3 import java.io.File;
4
5
6
7 public class ReadFile {
8     private File myFile;
9     private int n;
10    private int k;
11    private String nameFunction;
12    private String lastNameFunction;
13    private int recursion;
14    private boolean recursionBool;
15    private int nElevated;
16    private int openLoop;
17
18    public ReadFile (File myFile) {
19        this.myFile = myFile;
20        this.n = 0;
21        this.k = 0;
22        this.recursion = 0;
23        this.nElevated = 1;
24        this.openLoop = 0;
25        this.nameFunction = "";
26        this.recursionBool = false;
27    }
28    public void read() {}
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43    public void analyzer(String data) {}
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73    public void getFunctionName(String data) {}
74
75
76
77
78
79
80
81
82
83
84
85    public void result() {
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107 }
108

```

A contagem de operações é dinâmica, mas um precisamos que o usuário siga alguns padrões sugeridos, por exemplo, para definirmos uma análise que possui loops, é necessário que o coloque um comentário na linha *// openLoop*, para que o analisador interprete como um loop e *// closeLoop* para

que ele possa entender quando este loop encerra.

Neste analisador, toda linha que realizar um trabalho itera mais um de custo e recursões podem gerar uma complexidade $O(n \log n)$. Além do mais, este analisador resulta apenas o pior caso do código, sendo sempre a complexidade de notação O a ser impressa;

3. Análise Assintótica

A análise assintótica é uma ferramenta matemática que permite analisar o que ocorre com o comportamento de uma função $f(n)$ quando n tende ao infinito, para isso fazemos o uso de limitantes:

3.1 Notação O (Pior caso)

Sejam $f(n)$ e $g(n)$ duas funções de inteiros positivos. Dizemos que $f(n) = O(g(n))$, se existem constantes positivas C e n_0 , tais que:

$$f(n) \leq C g(n) \text{ para todo } n > n_0;$$

Logo, $f(n)$ é limitada superiormente. Assim, a notação fornece o limite superior para uma função $f(n)$.

Obs: $f(n)$ não cresce mais rápido que $C g(n)$ para $n > n_0$.

4. Conclusão

Por fim, a análise assintótica realizada pelo projeto nos fornece uma visão detalhada sobre o comportamento dos algoritmos de ordenação em C dando a complexidade de tempo baseado no custo das operações. Sendo assim, o projeto fornece de forma eficiente e precisa análise de algoritmos, mostrando qual código pode gerar menos custo de tempo ao usuário através de suas análises dando uma noção de qual melhor tipo de ordenação utilizar para grandes volumes dados.